

Подготовка к экзамену по C++ (ИТМО)

Тема: Вариативные шаблоны (Variadic Templates)

Hardcore Theory Edition

21 июня 2026 г.

1 Что такое Вариативные шаблоны?

До стандарта C++11, если вы хотели написать функцию, принимающую от 1 до 10 аргументов разных типов, вам приходилось писать 10 разных перегрузок этой функции (именно так был написан `boost::tuple`). Вариативные шаблоны решают эту проблему, позволяя передавать **произвольное количество аргументов** (Parameter Pack).

1.1 Синтаксис (Троеточие)

```
1 // Args - это Пакет Шаблонных Параметров (Template Parameter Pack)
2 // args - это Пакет Аргументов Функции (Function Parameter Pack)
3 template <typename... Args>
4 void print_all(Args... args) {
5     // Внутри args лежит от 0 до бесконечности переменных разных типов!
6 }
```

Троеточие **слева** от имени (`Args...`) означает *"Упаковать параметры в пакет"*. Троеточие **справа** от имени (`args...`) означает *"Распаковать пакет"*.

2 Как распаковать параметры? (C++11: Рекурсия)

Главная проблема: в C++ нет цикла `for`, чтобы пробежаться по пакету `args`. Пакет — это не массив, типы внутри него разные. Классический способ (до C++17) — это **Рекурсия на этапе компиляции**.

Мы пишем функцию, которая откусывает **один** (первый) аргумент, обрабатывает его, а остальной "хвост" пакета передает сама в себя!

```
1 // 1. Базовый случай (дно рекурсии) - когда аргументы закончились
2 void print() { std::cout << "\n"; }
3
4 // 2. Рекурсивный случай (откусываем Первый, остальное - Хвост)
5 template <typename First, typename... Rest>
6 void print(First first, Rest... rest) {
7     std::cout << first << " "; // Печатаем первый
8     print(rest...);           // Передаем хвост дальше (распаковка)
9 }
```

Суть на пальцах (Жизненная аналогия)

Представь, что пакет параметров — это матрешка. Ты не можешь засунуть руку внутрь и достать самую маленькую куколку. Тебе нужно снять верхнюю крышку (аргумент **First**), а всё остальное (аргумент **Rest...**) передать следующему человеку, чтобы он сделал то же самое, пока матрешки не кончатся (вызов пустой функции).

3 Выражения свертки (Fold Expressions, C++17)

Рекурсия — это сложно и долго компилируется. В C++17 добавили **Выражения свертки**, которые позволяют применить оператор (например, +, *, «) ко всем элементам пакета сразу **без рекурсии!**

Синтаксис: (args op ...) или (... op args)

```
1 template <typename... Args>
2 int sum_all(Args... args) {
3     // Распакует в: return (arg1 + (arg2 + arg3));
4     return (args + ...);
5 }
6
7 template <typename... Args>
8 void print_all_modern(Args... args) {
9     // Используем оператор запятая, чтобы вывести все элементы!
10    // Распакует в: (std::cout « arg1), (std::cout « arg2) ...
11    ((std::cout << args << " "), ...);
12 }
```

4 Оператор sizeof...

Если вам нужно узнать, сколько всего аргументов сейчас лежит в пакете, используйте специальный оператор (пишется с троеточием): `sizeof...(args)`. Он вычисляется на этапе компиляции.